SIMATS ENGINEERING

Department of _____

ASSIGNMENT 4 – CO4

Designing Reusable Class Hierarchies

Total Marks: 50

Course: Object Oriented Programming with C++
Course Code: DSA01

Student Name	: Krishnan Gupta M
Register Number	: 192524062
Faculty Name	: Arun Kumar
Submission Date	: 08 November 2026
GitHub Repository	: https://github.com/192524062simats-ops/DSA0103c-/tree/main/c%2B%2B%20assignment

TABLE OF CONTENTS

Task 1 – Class Design Review: Diagnose and Repair Duplicated Classes (6 Marks).....3
Task 2 – From Requirement to Hierarchy: University People Management (8 Marks).....8
Task 3 – Constructors in Derived Classes and Member Classes (9 Marks).....16
Task 4 – Pointers to Derived Classes: Forensics (7 Marks).....26
Task 5 – Pointers to Objects and the this Pointer (7 Marks).....34
Task 6 – Design Challenge: Vehicle Rental Fleet (13 Marks).....39
Overall Learning Outcomes.....50
Final Conclusion.....50
GitHub Repository Link.....50
References.....50

Task 1 – Class Design Review: Diagnose and Repair Duplicated Classes (6 Marks)

1.1 Aim

To diagnose the design problems in the duplicated Professor, LabAssistant and GuestFaculty payroll classes, and to repair the design as a class hierarchy with a common base class, protected data and a pay() function overridden in each derived class.

1.2 Problem Description

A college payroll program has three separate classes written by different developers. Each class repeats its own fields and pay rule, uses public data members and different names for the same information, and the three types cannot be processed together. The classes must be redesigned so that all staff can be handled in one loop and a new staff category is easy to add.

1.3 Task 1.1 – Diagnose the Design (2 Marks)

No.	Deficiency	Why it is a problem
1	Repeated fields and repeated pay rules	Professor and LabAssistant both declare name, id, basic and da and both repeat the formula basic + basic * 0.42. A change to the rule or a field must be made in several classes and can easily be missed in one of them.
2	Public data members	Any part of the program can change salary data directly (for example set basic to a negative value), so the classes cannot protect or validate their own data.
3	Inconsistent names for the same information	GuestFaculty uses nm and code for what the other classes call name and id. Code that reads these fields has to be written separately for each class.
4	Unused da field	da is declared in Professor and LabAssistant but is never used, because pay() uses a fixed 0.42 instead. It wastes memory and misleads the reader about how the allowance is calculated.
5	Magic number 0.42	The dearness allowance rate is typed directly inside two pay() functions with no name. Its meaning is not clear, and changing the rate means finding every place it appears.
6	Effort needed to add a new staff category	A new category means writing another complete class and copying the common fields and code again, so the amount of duplicated code keeps growing.
7	Cannot process all staff in one loop	The three classes have no common type, so they cannot be stored in one array or vector. Payroll needs a separate loop (or separate code) for every class.

1.4 Task 1.2 – Repair the Design (3 Marks)

The three classes now share a common abstract base class Staff. Staff holds the data every category needs (name, id and basic) as protected members, so derived classes can use it but outside code cannot change it, and it declares pay() as a pure virtual function. Each derived class overrides pay() with its own rule: Professor adds the research allowance (basic + basic * 0.42 + research), LabAssistant uses basic + basic * 0.42, and GuestFaculty uses hours * rate and

passes 0 as basic because guest faculty are paid by the hour. The unused da field is removed. display() is written once in the base class and calls pay() virtually, and the virtual destructor makes deletion through a Staff pointer safe.

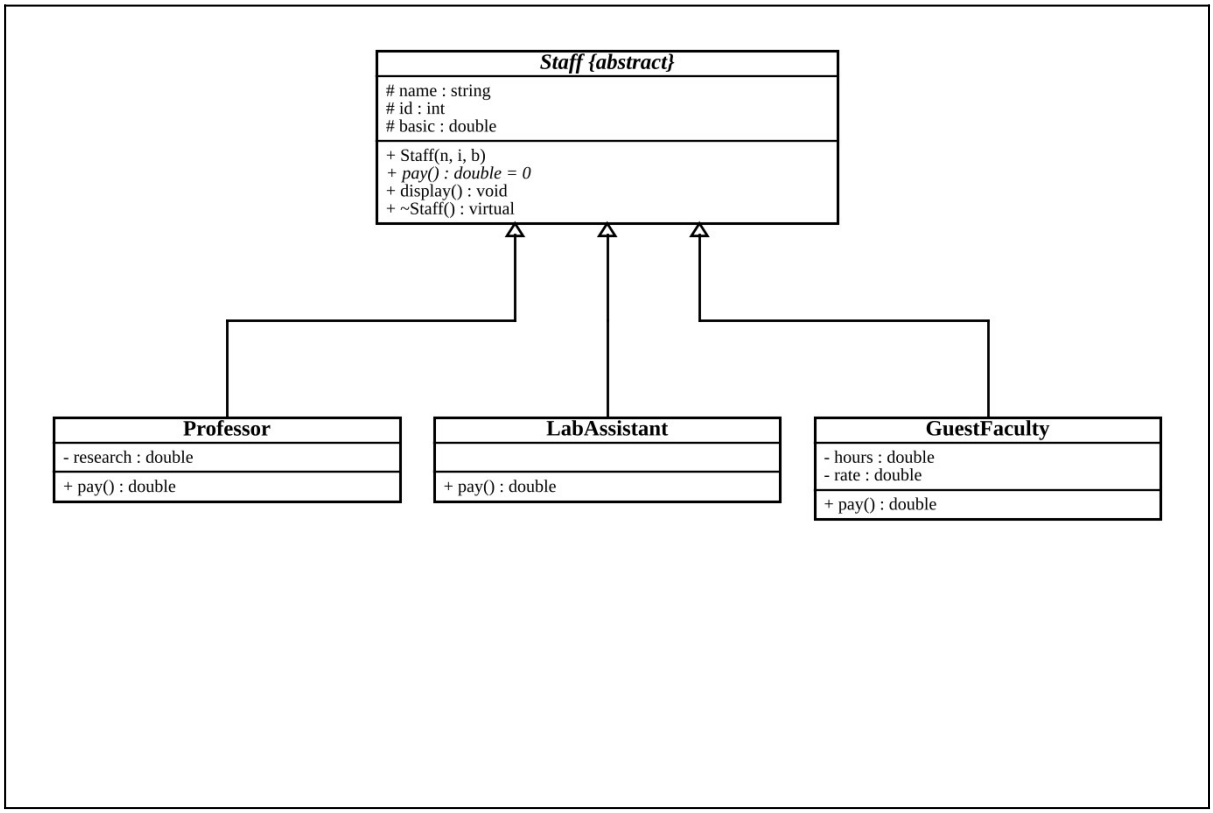


Figure 1.1: UML class diagram of the staff hierarchy

Complete C++ program for Task 1.2 (Task01.cpp):

```
#include <iostream>
#include <vector>
#include <memory>
#include <string>
using namespace std;

class Staff {
protected:
    string name;
    int id;
    double basic;

public:
    Staff(string n, int i, double b)
        : name(n), id(i), basic(b) {}

    virtual double pay() const = 0;

    virtual void display() const {
        cout << "Name: " << name << endl;
        cout << "ID: " << id << endl;
        cout << "Salary: " << pay() << endl;
    }
```

```
    virtual ~Staff() = default;
};

class Professor : public Staff {
    double research;

public:
    Professor(string n, int i, double b, double r)
        : Staff(n, i, b), research(r) {}

    double pay() const override {
        return basic + basic * 0.42 + research;
    }
};

class LabAssistant : public Staff {
public:
    LabAssistant(string n, int i, double b)
        : Staff(n, i, b) {}

    double pay() const override {
        return basic + basic * 0.42;
    }
};

class GuestFaculty : public Staff {
    double hours, rate;

public:
    GuestFaculty(string n, int i, double h, double r)
        : Staff(n, i, 0), hours(h), rate(r) {}

    double pay() const override {
        return hours * rate;
    }
};

int main() {
    vector<unique_ptr<Staff>> staff;

    staff.push_back(make_unique<Professor>(
        "Arun", 101, 50000, 10000));

    staff.push_back(make_unique<LabAssistant>(
        "Kumar", 102, 30000));

    staff.push_back(make_unique<GuestFaculty>(
        "Ravi", 103, 40, 500));

    cout << "COLLEGE PAYROLL DETAILS\n\n";

    for (const auto& s : staff) {
        s->display();
        cout << "------------------\n";
    }

    return 0;
```

```
}
```

Expected output:

```
COLLEGE PAYROLL DETAILS

Name: Arun
ID: 101
Salary: 81000
------------------
Name: Kumar
ID: 102
Salary: 42600
------------------
Name: Ravi
ID: 103
Salary: 20000
------------------
```

1.5 Program Execution Screenshot

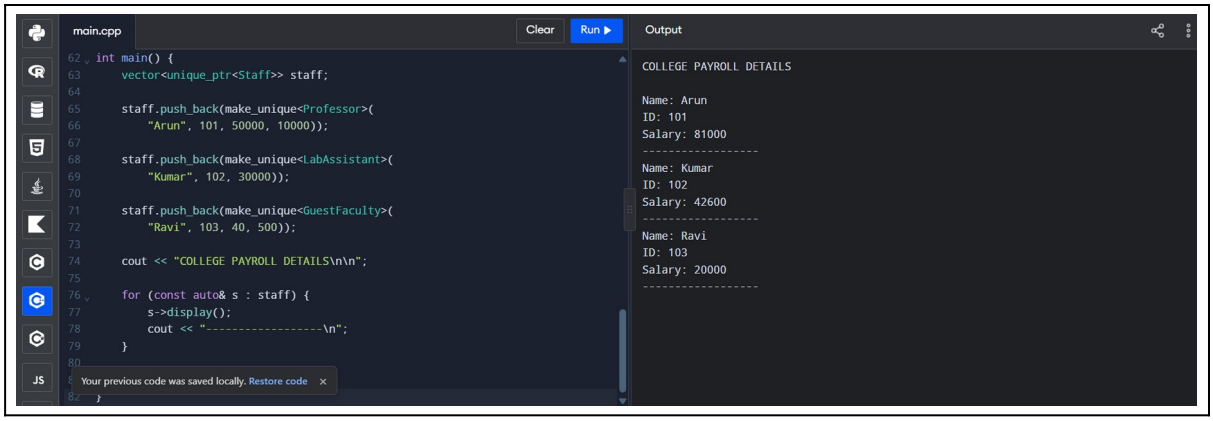


Figure 1.2: Output of the college payroll program

1.6 Output Explanation

Figure 1.2 shows the payroll details of three staff members printed from one loop over a vector of base-class smart pointers. Arun (Professor) gets 81000, Kumar (Lab Assistant) gets 42600 and Ravi (Guest Faculty) gets 20000. These match the pay rules: 50000 + 42% of 50000 + 10000 research = 81000, 30000 + 42% of 30000 = 42600, and 40 hours x 500 = 20000. The loop only knows about Staff, but the right pay() runs for each object because pay() is virtual.

1.7 Task 1.3 – Design Defence (1 Mark)

To add a new category, such as Researcher, I only write one new class derived from Staff and override pay(). The name, id and basic fields are inherited and display() is reused. The loop over vector<unique_ptr<Staff>> needs no change because it calls pay() through the base pointer. No existing class is edited, so working code is not disturbed, and the compiler makes sure the new class defines pay() because it is pure virtual.

1.8 Alternative Concept and Complexity Analysis

If the three staff types were represented by one Staff class with a type field and a switch inside pay(), the class would look like the code below.

Code / pseudocode of the alternative solution:

```
enum Type { PROF, LAB, GUEST };

class Staff {
    Type type;
    string name; int id;
    double basic, research, hours, rate;   // some unused
public:
    double pay() const {
        switch (type) {
            case PROF:  return basic + basic * 0.42 + research;
            case LAB:   return basic + basic * 0.42;
            case GUEST: return hours * rate;
        }
        return 0;
    }
};
```

Aspect	Original solution	Alternative solution
Time per pay() call	O(1): one virtual call through the vtable.	O(1): one switch (compare / jump).
Space per object	Each object stores only its own fields plus one hidden vtable pointer.	Every object stores the fields of all types (research, hours, rate) and a type code, so many fields are unused.
Places to change for a new staff type	One: write a new derived class (and create the object in main).	Several: the enum, the constructor, the switch in pay(), display() and any other switch on type, and all of the old code must be retested.

1.9 Conclusion

The duplicated classes had seven design problems. A Staff base class with a virtual pay() removes the repeated code, protects the data and allows all staff to be processed in one loop. The program prints the expected salaries for all three categories.

Task 2 – From Requirement to Hierarchy: University People Management (8 Marks)

2.1 Aim

To analyse the relationships in a university people-management system, design its class hierarchy, and use virtual base classes to solve the diamond problem of the TeachingAssistant class.

2.2 Problem Description

A university keeps records of persons (name, ID, phone). Students have a programme and CGPA, and staff have a department and salary. A teaching assistant is both a student and a staff member. Research scholars are students who also hold a fellowship.

2.3 Task 2.1 – Identify the Relationships (2 Marks)

Relationship	**is-a / has-a**	**Type of inheritance**
Student and Person	is-a	Single inheritance
Staff and Person	is-a	Single inheritance (Student and Staff together form hierarchical inheritance from Person)
TeachingAssistant and Student / Staff	is-a (both)	Multiple inheritance
ResearchScholar and Student	is-a	Single inheritance (Person -> Student -> ResearchScholar is multilevel)
ResearchScholar and Fellowship	has-a	Member class (no inheritance)
Complete hierarchy		Hybrid inheritance (hierarchical + multiple + multilevel, with a diamond)

2.4 Task 2.2 – Resolve the Diamond (2 Marks)

If TeachingAssistant inherits from Student and Staff, and both of them inherit from Person in the normal way, a TeachingAssistant object contains two separate Person sub-objects, one through Student and one through Staff. This causes three problems. The Person constructor runs twice (the output is Person Student Person Staff TA). The members name, id and phone are stored twice and can hold different values. Writing ta.name is ambiguous, and the compiler reports "request for member is ambiguous".

When Student and Staff inherit Person as a virtual base class, all inheritance paths share one Person sub-object. It is constructed only once, by the most-derived class (TeachingAssistant), before Student and Staff, so the output becomes Person Student Staff TA and ta.name is no longer ambiguous.

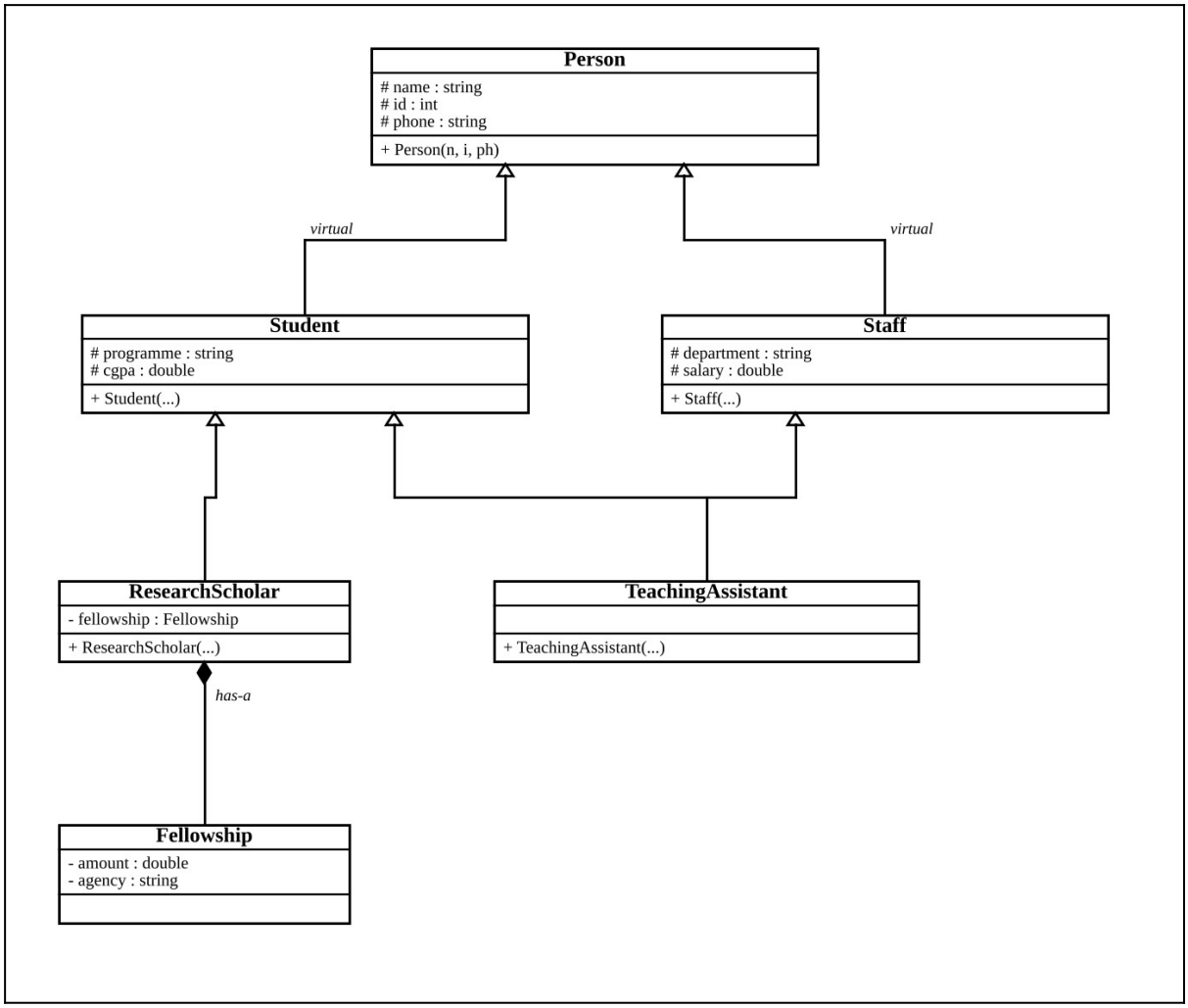


Figure 2.1: UML class diagram of the university hierarchy (virtual base class Person)

2.5 Task 2.3 – Design the Classes (3 Marks)

All the base classes are inherited publicly (Person by Student and Staff, Student by TeachingAssistant and ResearchScholar, Staff by TeachingAssistant), because every one of these is an is-a relationship: a Student can be used wherever a Person is expected. Protected or private inheritance would hide the Person interface from outside code and would mean "implemented in terms of", which is not what the requirement describes. The data members are protected so derived classes can use them, and Fellowship is a private member class of ResearchScholar (has-a).

Class declarations (compiles with g++ -std=c++17 -Wall -Wextra) (Task02_Design.cpp):

```
#include <string>
using namespace std;

class Person {
protected:
    string name;
    int id;
    string phone;

public:
    Person(string n, int i, string p)
```

```
        : name(n), id(i), phone(p) {}
};

class Student : virtual public Person {
protected:
    string programme;
    double cgpa;

public:
    Student(string n, int i, string p, string prog, double g)
        : Person(n, i, p), programme(prog), cgpa(g) {}
};

class Staff : virtual public Person {
protected:
    string department;
    double salary;

public:
    Staff(string n, int i, string p, string d, double s)
        : Person(n, i, p), department(d), salary(s) {}
};

class TeachingAssistant : public Student, public Staff {
public:
    TeachingAssistant(string n, int i, string p,
                      string prog, double g,
                      string d, double s)
        : Person(n, i, p),
          Student(n, i, p, prog, g),
          Staff(n, i, p, d, s) {}
};

class Fellowship {
    double amount;
    string agency;

public:
    Fellowship(double a, string ag)
        : amount(a), agency(ag) {}
};

class ResearchScholar : public Student {
    Fellowship fellowship;              // has-a (member class)

public:
    ResearchScholar(string n, int i, string p,
                    string prog, double g,
                    double amt, string ag)
        : Person(n, i, p),
          Student(n, i, p, prog, g),
          fellowship(amt, ag) {}
};
```

2.6 Task 2.4 – Access Table (1 Mark)

Member of Person	Public inheritance	Protected inheritance	Private inheritance
public member	public in Student	protected in Student	private in Student
protected member	protected in Student	protected in Student	private in Student
private member	not accessible	not accessible	not accessible

2.7 Predict the Output

My predicted output (written before running the program):

```
// Write your predicted output here BEFORE running the program.
```

Program with virtual base classes (Task02.cpp):

```
#include <iostream>
using namespace std;

class Person {
public:
    Person() {
        cout << "Person ";
    }
};

class Student : virtual public Person {
public:
    Student() {
        cout << "Student ";
    }
};

class Staff : virtual public Person {
public:
    Staff() {
        cout << "Staff ";
    }
};

class TeachingAssistant : public Student, public Staff {
public:
    TeachingAssistant() {
        cout << "TA ";
    }
};

int main() {
    TeachingAssistant ta;
```

```
    cout << endl;

    cout << "Size: " << sizeof(ta) << endl;

    return 0;
}
```

Expected output (the size depends on the compiler):

```
Person Student Staff TA
Size: 16
```

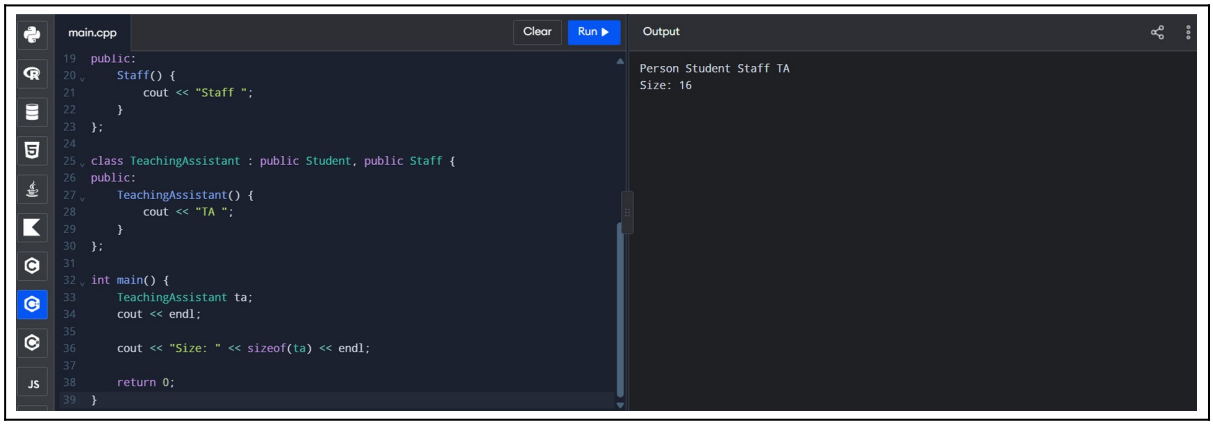


Figure 2.2: Actual output with virtual base classes (the program also prints sizeof(TeachingAssistant))

Same program without virtual (Task02_Diamond) (Task02_Diamond.cpp):

```
#include <iostream>
using namespace std;

class Person {
public:
    Person() {
        cout << "Person ";
    }
};

class Student : public Person {
public:
    Student() {
        cout << "Student ";
    }
};

class Staff : public Person {
public:
    Staff() {
        cout << "Staff ";
    }
};

class TeachingAssistant : public Student, public Staff {
public:
    TeachingAssistant() {
        cout << "TA ";
    }
};

int main() {
    TeachingAssistant ta;
    cout << endl;
    return 0;
}
```

Expected output:

```
Person Student Person Staff TA
```

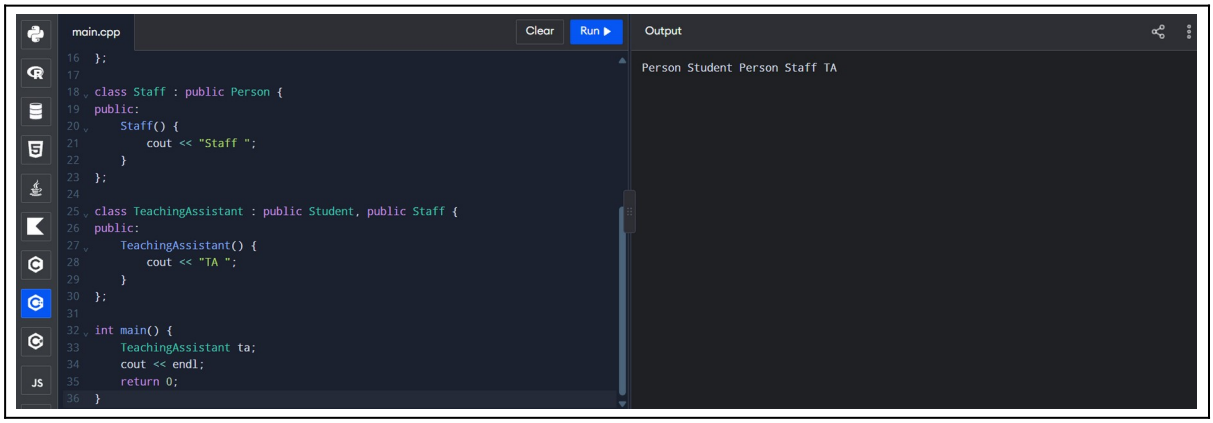


Figure 2.3: Actual output after removing the word virtual from the base classes

2.8 Output Explanation

In Figure 2.2 the output is "Person Student Staff TA" and the size is 16. Person is printed only once because Student and Staff share one virtual Person sub-object, and the constructor of a virtual base is called only by the most-derived class, TeachingAssistant, before the other bases. The Person() calls inside Student and Staff are skipped. Person has no data, so the 16 bytes are the two hidden pointers (one in Student and one in Staff) that lead to the shared virtual base. The exact number depends on the compiler.

In Figure 2.3, after removing virtual, the output is "Person Student Person Staff TA": Person is constructed twice because TeachingAssistant now holds two Person sub-objects. No compiler message appears with g++ -std=c++17 -Wall -Wextra, because the program never uses a member of Person. If it used ta.name or ta.id, the compiler would stop with the error "request for member is ambiguous". With g++ 13 on 64-bit Linux, sizeof(ta) without virtual prints 2 because Person is empty and the two sub-objects need different addresses.

Write here every difference between your prediction and the actual output, if there is any.

2.9 Alternative Concept and Complexity Analysis

If TeachingAssistant contained a Student object and a Staff object as members (composition, has-a) instead of inheriting from both through virtual base classes, the class would look like the code below.

Code / pseudocode of the alternative solution:

```
class TeachingAssistant {
    Student asStudent;      // has its own Person
    Staff   asStaff;        // has its own Person
public:
    TeachingAssistant(/* values */)
        : asStudent(/* ... */), asStaff(/* ... */) {}
};

// name must be reached through a member:
// ta.asStudent.name   or   ta.asStaff.name
```

Aspect	Original solution	Alternative solution
Space per object (sizeof)	One shared Person sub-object plus hidden virtual-base pointers (16 bytes in {v} with an empty Person).	A full Student and a full Staff, so two complete Person copies (name, id, phone stored twice).
Time to reach name	O(1): ta.name, with one extra indirection through the virtual base pointer.	O(1): ta.asStudent.name, a direct member access with no indirection.
Ambiguity	None: there is only one name field.	No compiler ambiguity, but there are two separate names that may hold different values, and the code must choose one.
Substitutability	A TeachingAssistant can be used as a Student, a Staff or a Person (polymorphism).	A TeachingAssistant is not a Student or a Staff, so it cannot be passed to code that expects them.

2.10 Conclusion

The relationships were classified as is-a or has-a and the hierarchy is hybrid. The diamond problem was shown by the output (Person constructed twice) and solved with virtual base classes, which gives one Person sub-object and the output Person Student Staff TA.

Task 3 – Constructors in Derived Classes and Member Classes (9 Marks)

3.1 Aim

To write constructors for derived classes and member classes using initializer lists, and to observe and explain the order of construction and destruction.

3.2 Problem Description

Student, Staff and TeachingAssistant pass values to their base classes through initializer lists. An Address member class is added to Person and a Course member class to Student. A message is printed in every constructor and destructor to trace the order of construction and destruction.

3.3 Task 3.1 – Pass Values to Base Constructors (3 Marks)

Each constructor passes the values its base class needs through its initializer list: Student(...) : Person(n, i, city, pin), ... and Staff(...) : Person(n, i, city, pin), ... TeachingAssistant calls Person(...), Student(...) and Staff(...) in its own list.

The most-derived class must call the virtual base constructor directly because the virtual Person is shared by Student and Staff, so neither of them can be the one that builds it. The C++ rule is that the most-derived class initializes every virtual base, and the Person(...) calls inside Student and Staff are ignored when a TeachingAssistant is built. If TeachingAssistant did not call Person(...) and Person has no default constructor, the program would not compile.

3.4 Task 3.2 – Member Classes (2 Marks)

Address is a member of Person and Course is a member of Student. Their constructors receive their values from the initializer list of the enclosing class: addr(city, pin) in Person and course(ct, cr) in Student. Members are constructed after the base classes and before the body of the constructor runs.

Program for Tasks 3.1, 3.2 and 3.3 (Task03_Hierarchy.cpp):

```
#include <iostream>
#include <string>
using namespace std;

class Address {
    string city;
    int pin;

public:
    Address(string c, int p) : city(c), pin(p) {
        cout << "Address ";
    }
    ~Address() { cout << "~Address "; }
};

class Course {
    string title;
    int credits;

public:
    Course(string t, int c) : title(t), credits(c) {
        cout << "Course ";
    }
```

```
    ~Course() { cout << "~Course "; }
};

class Person {
protected:
    string name;
    int id;
    Address addr;                       // member class

public:
    Person(string n, int i, string city, int pin)
        : name(n), id(i), addr(city, pin) {
        cout << "Person ";
    }
    ~Person() { cout << "~Person "; }
};

class Student : virtual public Person {
protected:
    string programme;
    double cgpa;
    Course course;                      // member class

public:
    Student(string n, int i, string city, int pin,
            string prog, double g, string ct, int cr)
        : Person(n, i, city, pin),
          programme(prog), cgpa(g), course(ct, cr) {
        cout << "Student ";
    }
    ~Student() { cout << "~Student "; }
};

class Staff : virtual public Person {
protected:
    string dept;
    double salary;

public:
    Staff(string n, int i, string city, int pin,
          string d, double s)
        : Person(n, i, city, pin), dept(d), salary(s) {
        cout << "Staff ";
    }
    ~Staff() { cout << "~Staff "; }
};

class TeachingAssistant : public Student, public Staff {
public:
    TeachingAssistant(string n, int i, string city, int pin,
                      string prog, double g,
                      string ct, int cr,
                      string d, double s)
        : Person(n, i, city, pin),     // virtual base: called directly
          Student(n, i, city, pin, prog, g, ct, cr),
          Staff(n, i, city, pin, d, s) {
        cout << "TA ";
```

```
    }
    ~TeachingAssistant() { cout << "~TA "; }
};

int main() {
    {
        cout << "Student : ";
        Student s("Kavya", 1, "Salem", 636001,
                  "BE CSE", 8.5, "OOP", 4);
        cout << "| ";
    }
    cout << endl;

    {
        cout << "Staff   : ";
        Staff f("Arun", 2, "Chennai", 600001,
                "CSE", 50000);
        cout << "| ";
    }
    cout << endl;

    {
        cout << "TA      : ";
        TeachingAssistant t("Ravi", 3, "Erode", 638001,
                            "ME CSE", 9.0, "DSA", 3,
                            "CSE", 20000);
        cout << "| ";
    }
    cout << endl;

    cout << "sizeof(TeachingAssistant) = "
         << sizeof(TeachingAssistant) << endl;
    return 0;
}
```

Expected output (the size depends on the compiler):

```
Student : Address Person Course Student | ~Student ~Course ~Person ~Address
Staff   : Address Person Staff | ~Staff ~Person ~Address
TA      : Address Person Course Student Staff TA | ~TA ~Staff ~Student
~Course ~Person ~Address
sizeof(TeachingAssistant) = 216
```

3.5 Task 3.3 – Trace the Order (3 Marks)

Figure 3.1
Construction and destruction order for Student, Staff and TeachingAssistant
[Insert screenshot here: Task3_Order_Trace.png]

Figure 3.1: Order of construction and destruction for the three objects (output of Task03_Hierarchy.cpp)

Object	Order of construction	Order of destruction
Student	Address, Person, Course, Student	~Student, ~Course, ~Person, ~Address
Staff	Address, Person, Staff	~Staff, ~Person, ~Address
TeachingAssistant	Address, Person, Course, Student, Staff, TA	~TA, ~Staff, ~Student, ~Course, ~Person, ~Address

Rule: virtual base classes are constructed first, then the other base classes in the order in which they are listed, then the member objects in the order in which they are declared, and finally the body of the derived class constructor runs. Destruction happens in exactly the reverse order. For TeachingAssistant, Person (with its Address) is built only once, before Student and Staff.

3.6 Task 3.4 – Experiment (1 Mark)

When virtual is removed from the base classes, TeachingAssistant contains two Person sub-objects. Its constructor can no longer call Person(...) directly (Person is no longer a virtual base), so that call is removed from its initializer list. The output then shows Address and Person twice, and the object becomes larger because the data of Person is stored twice.

Figure 3.2

Output and sizeof without virtual

[Insert screenshot here: Task3_NoVirtual_Sizeof.png]

Figure 3.2: Output and sizeof(TeachingAssistant) after removing virtual

Version	**Output for the TeachingAssistant object**	**sizeof(TeachingAssistant)**
With virtual	Address Person Course Student Staff TA | ~TA ~Staff ~Student ~Course ~Person ~Address	216
Without virtual	Address Person Course Student Address Person Staff TA | ~TA ~Staff ~Person ~Address ~Student ~Course ~Person ~Address	280

The sizes (216 and 280) were measured with g++ 13 on 64-bit Linux and depend on the compiler. Without virtual the object is larger here because a whole Person (with its Address) is stored twice.

3.7 Predict the Output

My predicted output (written before running the program):

```
// Write your predicted output here BEFORE running the program.
```

Prediction program (Task03.cpp):

```
#include <iostream>
using namespace std;

class Engine {
public:
    Engine() {
        cout << "Engine ";
```

```
    }

    ~Engine() {
        cout << "~Engine ";
    }
};

class Vehicle {
public:
    Vehicle() {
        cout << "Vehicle ";
    }

    ~Vehicle() {
        cout << "~Vehicle ";
    }
};

class Car : public Vehicle {
    Engine e;

public:
    Car() {
        cout << "Car ";
    }

    ~Car() {
        cout << "~Car ";
    }
};

int main() {
    {
        Car c;
        cout << "| ";
    }

    cout << endl;
    return 0;
}
```

Expected output:

```
Vehicle Engine Car | ~Car ~Engine ~Vehicle
```

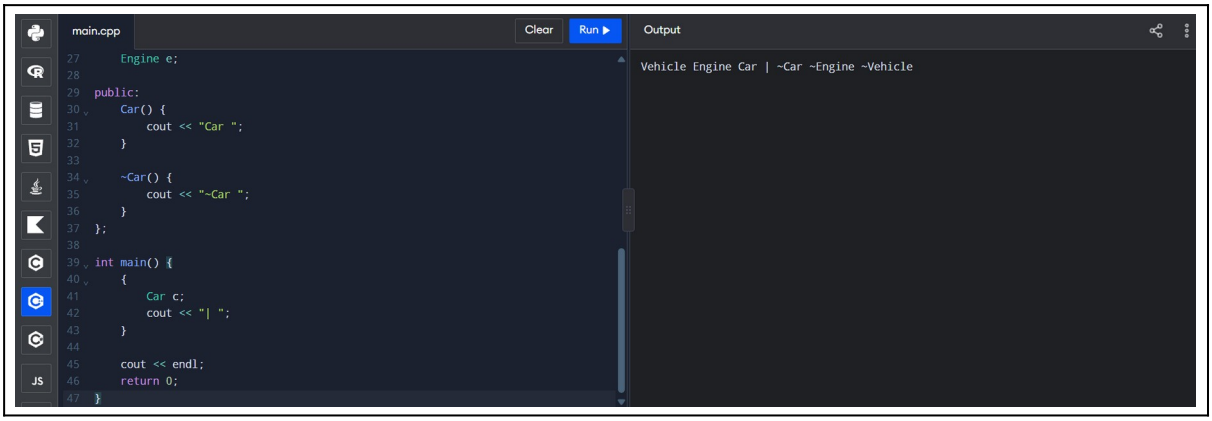


Figure 3.3: Actual output of the Car, Engine and Vehicle program

Figure 3.3 shows "Vehicle Engine Car | ~Car ~Engine ~Vehicle". The base class Vehicle is constructed first, then the member object Engine, then the body of the Car constructor runs and Car prints its name. The "|" is printed while c is alive. At the end of the block the destructors run in the reverse order: the body of ~Car, then the member Engine, then the base class Vehicle.

Rule: base classes are constructed first, then member objects (in declaration order), then the derived class body; destruction is the exact reverse (derived body, members, bases).

Write here every difference between your prediction and the actual output, if there is any.

3.8 Complete the Code

TODO 1 is the Student constructor. It initializes the Person base, regNo and the home Address member in the initializer list, and the rest of the program is unchanged:

Completed program (Task03_Construction.cpp):

```
#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
    string name;
    int age;

public:
    Person(string n, int a) : name(n), age(a) {}
};

class Address {
    string city;
    int pin;

public:
    Address(string c, int p) : city(c), pin(p) {}

    string str() const {
        return city + "-" + to_string(pin);
    }
```

```
};

class Student : public Person {
    long long regNo;
    Address home;

public:
    Student(string n, int a, long long r,
            string c, int p)
        : Person(n, a), regNo(r), home(c, p) {}

    void print() const {
        cout << regNo << " " << name
             << " (" << age << ") "
             << home.str() << endl;
    }
};

int main() {
    Student s("Kavya", 19, 192311001,
              "Salem", 636001);

    s.print();
    return 0;
}
```

Expected output:

```
192311001 Kavya (19) Salem-636001
```

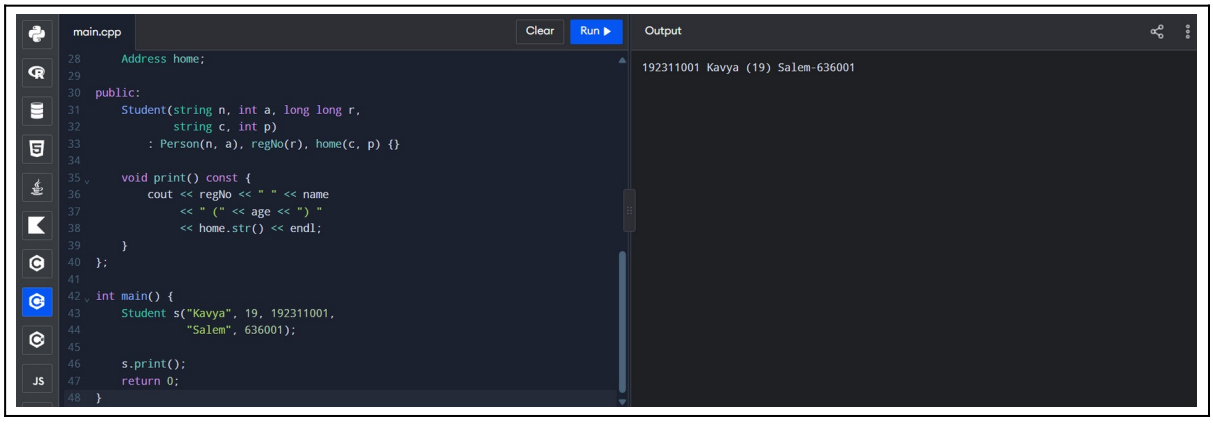


Figure 3.4: Output of the completed Student and Address program

Figure 3.4 prints "192311001 Kavya (19) Salem-636001", which matches the expected output.

Figure 3.5

Warning after swapping the initializer list order

[Insert screenshot here: Task3_Wreorder_Warning.png]

Figure 3.5: Compiler warning (-Wall) after swapping the order of the initializer list

After swapping regNo(r) and home(c, p) in the initializer list, g++ -Wall reports a warning of the form: "'Student::home' will be initialized after 'long long int Student::regNo' [-Wreorder] ... when initialized here". Members are always initialized in the order in which they are declared in the class (regNo first, then home), not in the order written in the list. The warning shows that the list does not match the real order, which can cause a bug if one member uses another member that is not yet initialized.

3.9 Alternative Concept and Complexity Analysis

If Student held a pointer to a dynamically created Address (Address* home = new Address(...)) instead of an Address member object, the class would look like the code below.

Code / pseudocode of the alternative solution:

```
class Student : public Person {
    long long regNo;
    Address* home;                      // pointer
public:
    Student(string n, int a, long long r, string c, int p)
        : Person(n, a), regNo(r), home(new Address(c, p)) {}

    Student(const Student& o)           // copy constructor (deep copy)
        : Person(o), regNo(o.regNo), home(new Address(*o.home)) {}

    Student& operator=(const Student& o) {
        if (this != &o) {
            Person::operator=(o);
            regNo = o.regNo;
            *home = *o.home;
        }
        return *this;
    }

    ~Student() { delete home; }         // destructor
};
```

Aspect	Original solution	Alternative solution
Extra time	None: the Address is built inside the Student.	One extra heap allocation (new) when a Student is created and one delete when it is destroyed. Each is O(1) but slower than a normal member.
Extra space	The Address sits inside the Student object.	A pointer (8 bytes) in the Student plus the Address block on the heap and the allocator overhead.
Constructor	Initializes home(c, p) in the list.	Allocates with new Address(c, p).
Copy constructor / assignment	The compiler-generated ones copy the member correctly.	Must make a deep copy (new Address(*o.home)); a shallow copy would make two Students share one Address and delete it twice.
Destructor	Nothing to do for home.	Must delete home, otherwise the memory leaks.

3.10 Conclusion

The derived-class constructors pass values to their bases through initializer lists, and the traced output confirms the rule: virtual bases first, then other bases, then members, then the constructor body, with destruction in reverse order. The experiment showed that virtual changes both the output and the size of the object.

Task 4 – Pointers to Derived Classes: Forensics (7 Marks)

4.1 Aim

To find and prove the defects in a bank account program that uses base-class pointers (wrong interest rate, memory leak and object slicing), and to repair it using virtual functions and a virtual destructor.

4.2 Problem Description

A bank stores different kinds of accounts through base-class pointers. Senior-citizen accounts should earn 4.0% interest, but every account earns 3.5%, and memory reports show that part of each senior account is never released.

4.3 Task 4.1 – Locate the Defects (2 Marks)

Defect	Cause	Effect
Senior rate is never used	rate() is not virtual in Account. A call through an Account* is decided at compile time from the pointer type, so Account::rate runs. SeniorAccount::rate only hides it.	Every account earns 3.5%.
Memory leak	The destructor of Account is not virtual, so delete a on an Account* that really points to a SeniorAccount calls only ~Account. ~SeniorAccount never runs (this is undefined behaviour).	The history array (100 doubles = 800 bytes) of every senior account is never released.
Object slicing in the last line	Account copy = SeniorAccount() copies only the Account part of the senior account into an Account object.	The derived part (history, the 4.0 rate) is lost and the copy behaves as a plain Account.

4.4 Task 4.2 – Construct Evidence (2 Marks)

The original program was changed only to print a message in each destructor, and the commands used are:

```
g++ -std=c++17 -Wall -Wextra -g -fsanitize=address Task04_Defects.cpp -o
defects
./defects
```

Original program with destructor messages (Task04_Defects.cpp):

```
#include <iostream>
#include <vector>
using namespace std;

class Account {
public:
    double rate() const { return 3.5; }          // not virtual
    ~Account() { cout << "Account destroyed\n"; } // not virtual
};

class SeniorAccount : public Account {
    double* history = new double[100];
public:
    double rate() const { return 4.0; }
    ~SeniorAccount() {
        delete[] history;
        cout << "Senior Account destroyed\n";
    }
};

int main() {
    Account* accounts[2] = { new Account, new SeniorAccount };

    for (Account* a : accounts)
        cout << "Interest Rate: " << a->rate() << "%\n";

    for (Account* a : accounts)
        delete a;                    // derived destructor never runs

    Account copy = SeniorAccount();  // object slicing
    cout << "Copy rate: " << copy.rate() << "%\n";
    return 0;
}
```

Expected output (without the sanitizer):

```
Interest Rate: 3.5%
Interest Rate: 3.5%
Account destroyed
Account destroyed
Senior Account destroyed
Account destroyed
Copy rate: 3.5%
Account destroyed
```

Figure 4.1
Output of the original program with destructor messages
[Insert screenshot here: Task4_Defect_Output.png]

Figure 4.1: Output of the original program showing the defects

Figure 4.2
AddressSanitizer report (-fsanitize=address)
[Insert screenshot here: Task4_AddressSanitizer.png]

Figure 4.2: AddressSanitizer report for the original program

How the evidence proves each defect: both accounts print "Interest Rate: 3.5%", so the senior rate is never used. In the delete loop the message "Account destroyed" is printed twice and "Senior Account destroyed" is never printed, so ~SeniorAccount does not run and its array is not released. The messages "Senior Account destroyed" and "Account destroyed" that appear after the loop belong to the temporary SeniorAccount() in the last line, and "Copy rate: 3.5%" shows that the copy is only an Account (slicing).

With -fsanitize=address, AddressSanitizer stops the program at the delete line with "new-delete-type-mismatch": the object was allocated as an 8-byte SeniorAccount but is deleted as a 1-byte Account. This is the sanitizer evidence for the missing virtual destructor. To see the leak itself, run with ASAN_OPTIONS=new_delete_type_mismatch=0; the report then says "Direct leak of 800 byte(s)", which is the history array.

4.5 Task 4.3 – Repair the Program (2 Marks)

rate() is made virtual in Account and overridden in SeniorAccount, and the destructor of Account is made virtual. The accounts are used only through pointers, so no copy of a SeniorAccount into an Account is made and slicing cannot happen.

Repaired program (Task04_Sanitizer.cpp):

```
#include <iostream>
using namespace std;

class Account {
public:
    virtual double rate() const {
        return 3.5;
    }

    virtual ~Account() {
        cout << "Account destroyed\n";
    }
};

class SeniorAccount : public Account {
    double* history;

public:
    SeniorAccount() {
        history = new double[100];
        cout << "Senior Account created\n";
    }

    double rate() const override {
        return 4.0;
    }

    ~SeniorAccount() override {
        delete[] history;
        cout << "Senior Account destroyed\n";
    }
};

int main() {
    Account* accounts[2];

    accounts[0] = new Account();
    accounts[1] = new SeniorAccount();

    for (Account* a : accounts) {
        cout << "Interest Rate: "
             << a->rate() << "%\n";
    }

    for (Account* a : accounts) {
        delete a;
    }

    return 0;
}
```

Expected output:

```
Senior Account created
Interest Rate: 3.5%
Interest Rate: 4%
Account destroyed
Senior Account destroyed
Account destroyed
```

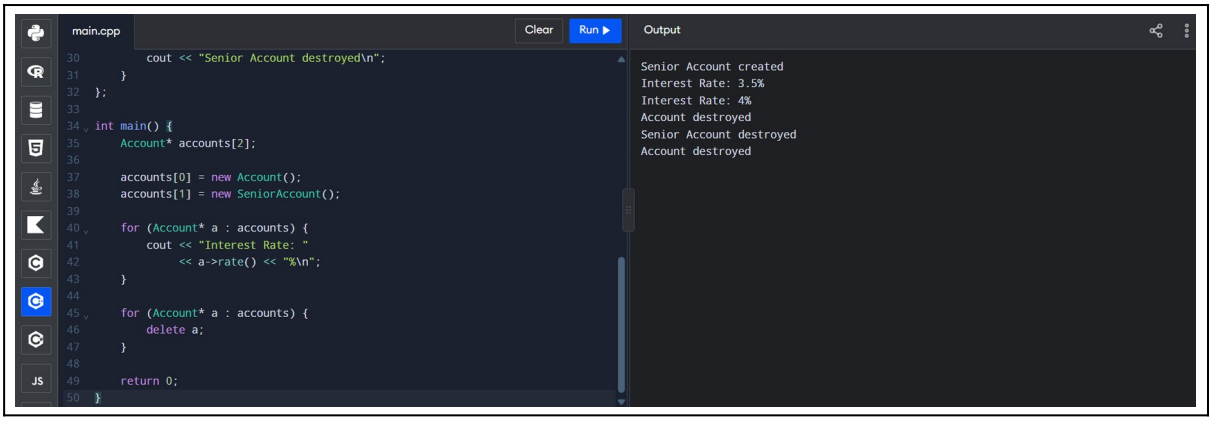


Figure 4.3: Output of the repaired program

Figure 4.3 shows 3.5% for the ordinary account and 4% for the senior account, so the correct rate() is now called through the base-class pointer. When the senior account is deleted, "Senior Account destroyed" is printed first and then "Account destroyed", so the derived part is released before the base part and the history array no longer leaks.

4.6 Task 4.4 – Prevent Recurrence (1 Mark)

Rule 1: A base class that has virtual functions must have a virtual destructor, so that deleting through a base pointer destroys the whole object.

Rule 2: Declare the function virtual in the base class and write override in every derived class, and compile with -Wall -Wextra, so that a function that does not really override (missing virtual or wrong signature) is reported by the compiler. Polymorphic objects should also be stored and passed by pointer, reference or smart pointer, never by value.

4.7 Predict the Output

My predicted output (written before running the program):

```
// Write your predicted output here BEFORE running the program.


```

Prediction program (Task04.cpp):

```
#include <iostream>
```

```
using namespace std;

class Account {
public:
    double rate() const {
        return 3.5;
    }

    virtual double vrate() const {
        return 3.5;
    }

    virtual ~Account() = default;
};

class SeniorAccount : public Account {
public:
    double rate() const {
        return 4.0;
    }

    double vrate() const override {
        return 4.0;
    }
};

int main() {
    SeniorAccount s;
    Account* p = &s;

    Account copy = s;

    cout << p->rate() << " "
         << p->vrate() << " "
         << copy.vrate() << endl;

    return 0;
}
```

Expected output:

```
3.5 4 3.5
```

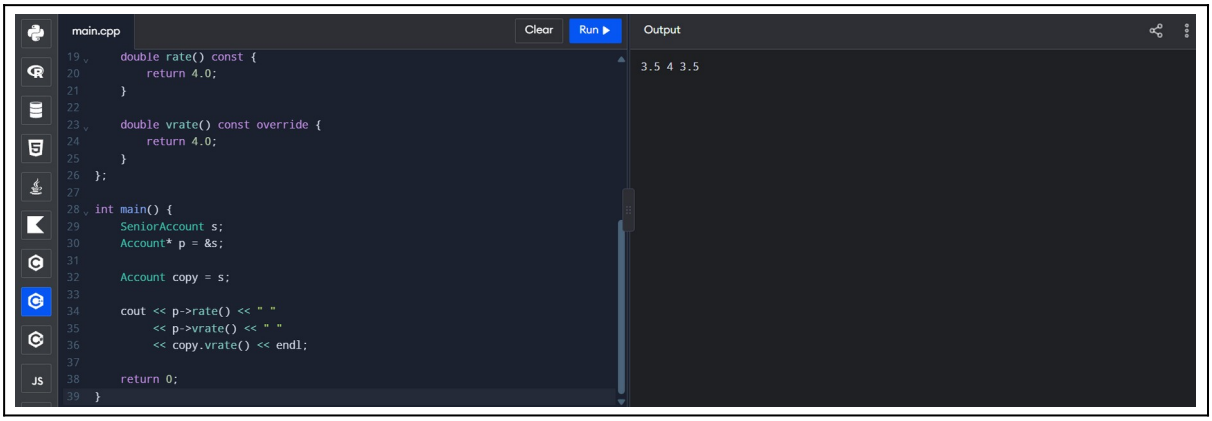


Figure 4.4: Actual output of the prediction program

Figure 4.4 shows "3.5 4 3.5". The first value is 3.5 because rate() is not virtual: the call p->rate() is bound at compile time using the pointer type Account*, so Account::rate runs. The second value is 4 because vrate() is virtual: the call is bound at run time using the real object (a SeniorAccount), so SeniorAccount::vrate runs. The third value is 3.5 because "Account copy = s" copies only the Account part of s (object slicing); copy is a real Account object, so copy.vrate() calls Account::vrate even though it is virtual.

Write here every difference between your prediction and the actual output, if there is any.

4.8 Alternative Concept and Complexity Analysis

If the accounts were stored as std::unique_ptr<Account> in a std::vector, instead of raw pointers deleted by hand, the code would look like the code below.

Code / pseudocode of the alternative solution:

```
vector<unique_ptr<Account>> accounts;
accounts.push_back(make_unique<Account>());
accounts.push_back(make_unique<SeniorAccount>());

for (const auto& a : accounts)
    cout << "Interest Rate: " << a->rate() << "%\n";

// no delete: the vector releases every account automatically
```

Aspect	Original solution	Alternative solution
Time	O(n) to process n accounts (one virtual call each).	O(n): the same. The cost of the loop does not change.
Space	n pointers (8 bytes each) plus the n objects on the heap.	The same: a unique_ptr is the same size as a raw pointer. Total O(n).
Wrong rate (non-virtual rate())	Can happen.	Can still happen: smart pointers do not change binding. rate() must still be virtual.
Memory leak	Happens when delete is forgotten or the destructor is not virtual.	Forgetting delete is no longer possible. But unique_ptr<Account> still deletes through Account*, so without a virtual destructor the derived part is still not destroyed.
Object slicing	Happens when a derived object is copied into a base object.	unique_ptr cannot be copied, which prevents copying the pointer, but slicing can still occur if the code copies the object itself (Account c = *a;).

4.9 Conclusion

The three defects were located, proved with destructor messages and AddressSanitizer, and repaired with a virtual rate(), a virtual destructor and pointers instead of copies. The prediction program shows the difference between compile-time and run-time binding.

Task 5 – Pointers to Objects and the this Pointer (7 Marks)

5.1 Aim

To access a Cart object directly, through a pointer, through a reference and through new and delete, and to use the this pointer for method chaining and for separating a member from a parameter with the same name.

5.2 Problem Description

A grocery delivery app keeps a shopping Cart object for each customer. The Cart must support chained add() and setDiscount() calls, and the app must find the cart with the highest total from an array of pointers to Cart objects.

5.3 Task 5.1 – Four Ways to Reach an Object (3 Marks)

The program uses a Cart (a) directly by name, (b) through a pointer with ->, (c) through a reference, and (d) through a pointer created with new and released with delete. add() and setDiscount() return *this, so calls can be chained, and setDiscount() uses this->discount to separate the member from the parameter that has the same name.

5.4 Task 5.2 – Change the Requirement (2 Marks)

The carts are kept in an array of four pointers. A loop with indexing keeps a pointer to the cart with the highest total() and compares each cart with it. The array holds the addresses of three stack carts and one heap cart, and the heap cart is deleted at the end.

Program for Tasks 5.1 and 5.2 (Task05_FourWays.cpp):

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class Cart {
    string items[10];
    double prices[10];
    int n = 0;
    double discount = 0;

public:
    Cart& add(string item, double price) {
        if (n < 10) {
            items[n] = item;
            prices[n] = price;
            n++;
        }
        return *this;                    // enables chaining
    }

    Cart& setDiscount(double discount) {
        this->discount = discount;       // member = parameter
        return *this;
    }

    double total() const {
        double t = 0;
```

```
        for (int i = 0; i < n; i++) t += prices[i];
        return t * (100 - discount) / 100;
    }
};

int main() {
    cout << fixed << setprecision(2);

    // (a) directly by name
    Cart a;
    a.add("Rice", 60).add("Dal", 120).setDiscount(10);
    cout << "(a) by name      : " << a.total() << endl;

    // (b) through a pointer with ->
    Cart b;
    Cart* pb = &b;
    pb->add("Oil", 180);
    cout << "(b) pointer ->   : " << pb->total() << endl;

    // (c) through a reference
    Cart c;
    Cart& rc = c;
    rc.add("Milk", 50).add("Bread", 40);
    cout << "(c) reference    : " << rc.total() << endl;

    // (d) pointer created with new, released with delete
    Cart* pd = new Cart;
    pd->add("Tea", 100).add("Sugar", 45).setDiscount(20);
    cout << "(d) new / delete : " << pd->total() << endl;
    delete pd;

    // Task 5.2: array of pointers, find the highest total
    Cart* carts[4] = { &a, &b, &c, new Cart };
    carts[3]->add("Soap", 30);

    Cart* best = carts[0];
    for (int i = 1; i < 4; i++) {
        if (carts[i]->total() > best->total())
            best = carts[i];
    }
    cout << "Highest total    : " << best->total() << endl;

    delete carts[3];
    return 0;
}
```

Expected output:

```
(a) by name      : 162.00
(b) pointer ->   : 180.00
(c) reference    : 90.00
(d) new / delete : 116.00
Highest total    : 180.00
```

5.5 Task 5.3 – Compare (2 Marks)

Way	Safety	Lifetime	When to use
Directly by name	Safe: the object always exists and cannot be null.	Ends at the end of the enclosing block (automatic).	When the object is needed only in the current scope.
Pointer with ->	Can be null or dangling, so it must be checked.	The pointer does not own the object, and it can be pointed at another object.	Arrays of objects, optional objects, and passing addresses to functions.
Reference	Safer than a pointer: it must be bound when created and cannot be null or reseated.	Same as the object it refers to; it dangles if the object is destroyed.	Function parameters, aliases, and returning *this for chaining.
Pointer from new / delete	Least safe: leak if delete is forgotten, double delete, dangling pointer.	Chosen by the programmer; the object survives the block.	When the object must outlive the scope or is created at run time (prefer smart pointers).

this is a pointer because a member function is given the address of the object it was called on as a hidden parameter, and this holds that address (of type Cart*). *this dereferences it, so *this is the calling object itself, and returning Cart& gives that same object back, which makes chaining possible.

5.6 Complete the Code

TODO 1 (add) stores the item and price when there is space and returns *this. TODO 2 (setDiscount) uses this->discount to tell the member from the parameter and returns *this.

Completed program (Task05.cpp):

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class Cart {
    string items[10];
    double prices[10];
    int n = 0;
    double discount = 0;

public:
    Cart& add(string item, double price) {
        if (n < 10) {
            items[n] = item;
            prices[n] = price;
            n++;
        }

        return *this;
    }
```

```
    Cart& setDiscount(double discount) {
        this->discount = discount;
        return *this;
    }

    double total() const {
        double t = 0;

        for (int i = 0; i < n; i++) {
            t += prices[i];
        }

        return t * (100 - discount) / 100;
    }
};

int main() {
    Cart c;

    c.add("Rice", 60)
     .add("Dal", 120)
     .setDiscount(10);

    Cart* p = &c;
    p->add("Oil", 180);

    cout << fixed << setprecision(2);
    cout << p->total() << endl;

    return 0;
}
```

Expected output:

```
324.00
```

5.7 Program Execution Screenshot

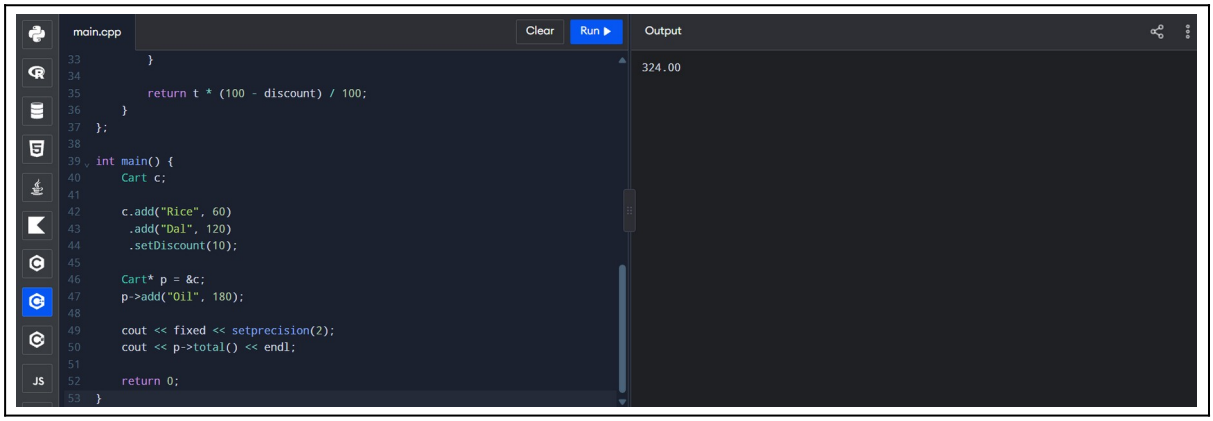


Figure 5.1: Output of the completed Cart program

Figure 5.1 prints 324.00, which matches the expected output. The three items cost 60 + 120 + 180 = 360 and a 10% discount gives 324.00. The call p->add("Oil", 180) works on the same object c through the pointer p.

If add() returned Cart instead of Cart&, each call would return a copy. In c.add("Rice", 60).add("Dal", 120).setDiscount(10), only the first add() works on c; the next calls work on temporary copies that are thrown away. c would hold only Rice and have no discount, and after p->add("Oil", 180) the total would be 60 + 180 = 240.00 instead of 324.00 (I checked this by compiling the change).

5.8 Alternative Concept and Complexity Analysis

If the carts were kept in a std::vector<Cart> (objects) instead of an array of pointers to Cart objects, the code would look like the code below.

Code / pseudocode of the alternative solution:

```
vector<Cart> carts(4);
carts[0].add("Rice", 60).setDiscount(10);
carts[3].add("Soap", 30);

size_t best = 0;
for (size_t i = 1; i < carts.size(); i++)
    if (carts[i].total() > carts[best].total())
        best = i;
```

Aspect	Original solution	Alternative solution
Time to find the highest total	O(n): one pass over n pointers.	O(n): one pass over n objects.
Space	One pointer (8 bytes) for each cart; the Cart objects are stored elsewhere and nothing is copied.	The whole Cart objects (10 strings, 10 prices and the counters) are stored inside the vector, and push_back copies them.
When the container grows	Pointers stay valid because the Cart objects do not move.	The vector may reallocate and move all the objects, so any pointer or reference into the vector becomes dangling. Indexes should be used instead.

5.9 Conclusion

The Cart class was used in four ways and compared on safety and lifetime. The this pointer allowed chaining and removed the name clash, and the program printed the expected total of 324.00.

Task 6 – Design Challenge: Vehicle Rental Fleet (13 Marks)

6.1 Aim

To design, implement and demonstrate a vehicle rental class hierarchy with an abstract base class and virtual functions, so that new vehicle types can be added without changing the existing classes.

6.2 Problem Description

A rental company rents bicycles, two-wheelers, cars and electric cars. All vehicles have a registration number and a daily rate. Motor vehicles also have a fuel or charge level and a service due date, cars have a seating capacity, and electric cars have a battery capacity and are charged per kWh. Each vehicle computes its rental charge differently. The company needs to rent, return and list vehicles and add new vehicle types in future without changing existing classes.

6.3 Stage 1 – Interpret the Problem (2 Marks)

Entity	Data	Behaviour that differs
Vehicle (common)	Registration number, daily rate, rented status	rent(), returnVehicle() and display() are common; charge() differs for every type
Bicycle	No extra data	charge = days x daily rate
Two-wheeler	Fuel level, service due date	charge = days x daily rate
Car	Fuel level, service due date, seating capacity	charge = days x daily rate
Electric car	Charge level, service due date, seats, battery capacity (kWh), rate per kWh	charge = days x daily rate + battery kWh x rate per kWh

Requirements as testable statements:

R1. Every vehicle has a registration number and a daily rate.

R2. Renting an available vehicle prints "<regNo> rented successfully"; renting a vehicle that is already rented is refused.

R3. Returning a rented vehicle makes it available again; returning a vehicle that was not rented is refused.

R4. A late return adds a late charge equal to late days x daily rate (for example 2 late days at 200 per day give 400).

R5. charge(days) for a bicycle, two-wheeler or car is days x daily rate (for example 2 days at 300 give 600).

R6. charge(days) for an electric car is days x daily rate + battery kWh x rate per kWh (for example 2 x 2000 + 75 x 2 = 4150).

R7. All vehicles can be kept in one container of base-class pointers and listed in one loop.

R8. A new vehicle type (an electric bus) can be added by writing one new class, without editing any existing class (for example 2 x 2500 + 200 x 5 = 6000).

6.4 Stage 2 – Propose the Hierarchy (2 Marks)

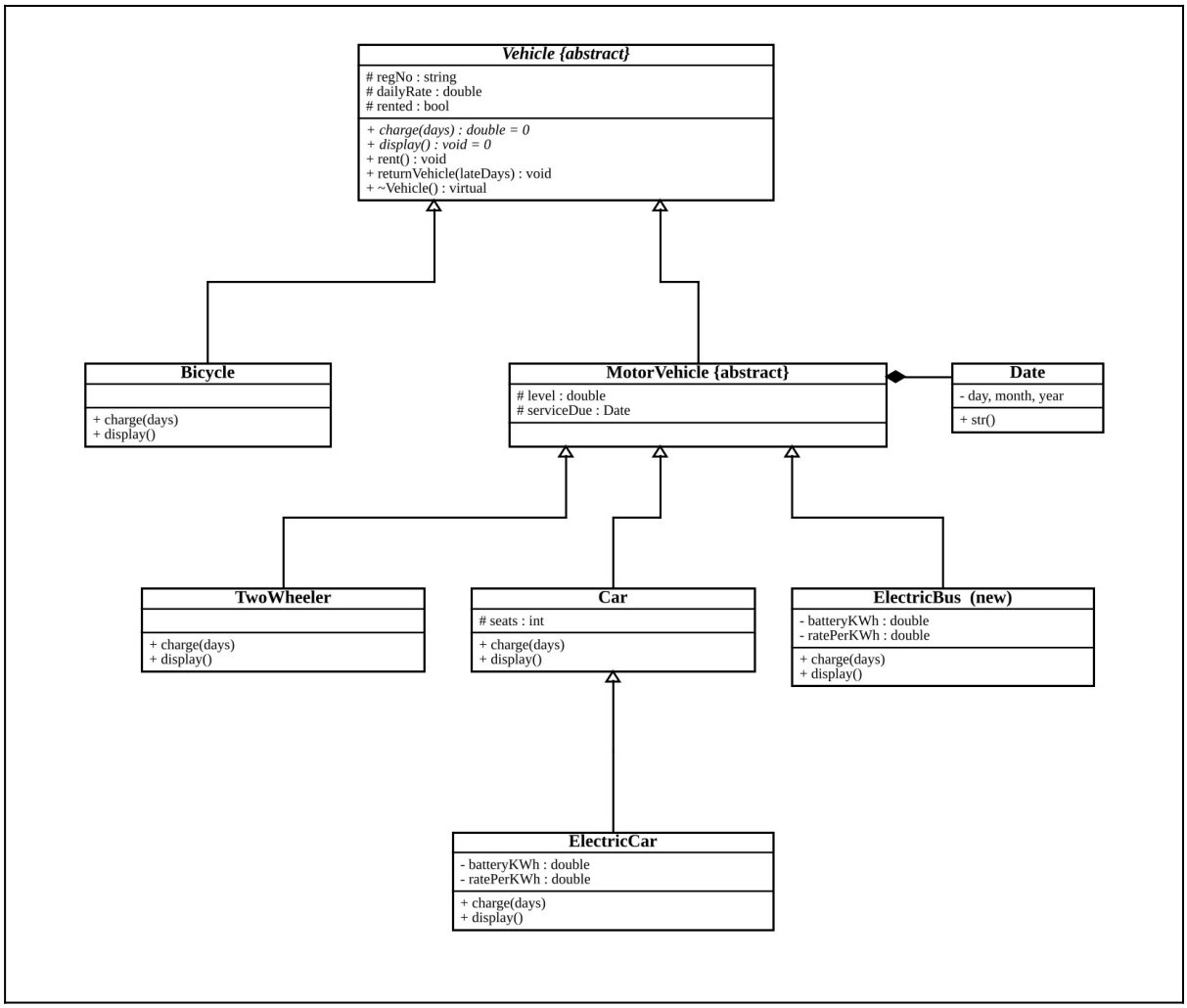


Figure 6.1: UML class diagram of the vehicle rental fleet

Vehicle is the abstract base class (it has the pure virtual functions charge() and display()). Bicycle and MotorVehicle derive from it, so the hierarchy is hierarchical inheritance. TwoWheeler, Car and ElectricBus derive from MotorVehicle (hierarchical), and ElectricCar derives from Car, which makes Vehicle -> MotorVehicle -> Car -> ElectricCar multilevel inheritance. All inheritance is public single inheritance. Date is a member class of MotorVehicle (the service due date, a has-a relationship).

6.5 Stage 3 – Identify the Critical Design Decisions (2 Marks)

Class	Members placed in it	Virtual functions
Vehicle	regNo, dailyRate, rented; rent() and returnVehicle() (the same for every vehicle, written once)	charge() and display() (pure virtual), virtual destructor
Bicycle	None	charge(), display()
MotorVehicle	level (fuel or charge), serviceDue (Date member class)	None added (still abstract)
TwoWheeler	None	charge(), display()
Car	seats	charge(), display()
ElectricCar	batteryKWh, ratePerKWh	charge() (adds the kWh cost), display()
ElectricBus (new type)	batteryKWh, ratePerKWh	charge(), display()

The fleet is stored as vector<unique_ptr<Vehicle>>, a container of base-class pointers, so objects of every type are kept together and used through virtual calls. rent() and returnVehicle() are not virtual because they are the same for all vehicles. The destructor is virtual so that deleting through a Vehicle pointer is safe.

6.6 Stage 4 – Design the Classes (4 Marks)

Class declarations:

```
class Date {                                // member class
    int day, month, year;
public:
    Date(int d = 1, int m = 1, int y = 2026);
    string str() const;
};

class Vehicle {                             // abstract base class
protected:
    string regNo;
    double dailyRate;
    bool rented = false;
public:
    Vehicle(string r, double rate);
    virtual double charge(int days) const = 0;
    virtual void display() const = 0;
    void rent();
    void returnVehicle(int lateDays);
    virtual ~Vehicle() = default;
};

class Bicycle : public Vehicle { ... charge(), display() ... };

class MotorVehicle : public Vehicle {
protected:
    double level;                           // fuel or charge level
    Date serviceDue;                        // member class
public:
    MotorVehicle(string r, double rate, double lv, Date d);
};

class TwoWheeler  : public MotorVehicle { ... charge(), display() ... };
class Car         : public MotorVehicle { int seats; ... };
class ElectricCar : public Car { double batteryKWh, ratePerKWh; ... };
class ElectricBus : public MotorVehicle { double batteryKWh,
ratePerKWh; ... };
```

Pseudocode for renting and returning a vehicle:

```
ALGORITHM RentVehicle(vehicle)
INPUT   : vehicle, a pointer to a Vehicle
OUTPUT  : a message
BEGIN
    IF vehicle.rented = TRUE THEN
        DISPLAY vehicle.regNo, "is already rented"
    ELSE
        SET vehicle.rented <- TRUE
        DISPLAY vehicle.regNo, "rented successfully"
    END IF
END

ALGORITHM ReturnVehicle(vehicle, lateDays)
INPUT   : vehicle, lateDays (days returned late)
OUTPUT  : a message and the late charge
BEGIN
    IF vehicle.rented = FALSE THEN
        DISPLAY vehicle.regNo, "was not rented"
    ELSE
        SET vehicle.rented <- FALSE
        DISPLAY vehicle.regNo, "returned"
        IF lateDays > 0 THEN
            SET lateCharge <- lateDays * vehicle.dailyRate
            DISPLAY "Late Charge:", lateCharge
        END IF
    END IF
END

ALGORITHM Charge(vehicle, days)          // chosen at run time (virtual)
BEGIN
    IF vehicle is an ElectricCar OR an ElectricBus THEN
        RETURN days * dailyRate + batteryKWh * ratePerKWh
    ELSE
        RETURN days * dailyRate
    END IF
END
```

6.7 Stage 5 – Implement and Demonstrate (3 Marks)

Complete C++ program (Task06.cpp):

```
#include <iostream>
#include <vector>
#include <memory>
#include <string>
using namespace std;

class Date {                                    // member class
    int day, month, year;

public:
    Date(int d = 1, int m = 1, int y = 2026)
        : day(d), month(m), year(y) {}
```

```
    string str() const {
        return to_string(day) + "/" + to_string(month) +
               "/" + to_string(year);
    }
};

class Vehicle {                                  // abstract base class
protected:
    string regNo;
    double dailyRate;
    bool rented = false;

public:
    Vehicle(string r, double rate)
        : regNo(r), dailyRate(rate) {}

    virtual double charge(int days) const = 0;
    virtual void display() const = 0;

    void rent() {
        if (rented) {
            cout << regNo << " is already rented\n";
            return;
        }
        rented = true;
        cout << regNo << " rented successfully\n";
    }

    void returnVehicle(int lateDays) {
        if (!rented) {
            cout << regNo << " was not rented\n";
            return;
        }
        rented = false;
        cout << regNo << " returned\n";

        if (lateDays > 0)
            cout << "Late Charge: "
                 << lateDays * dailyRate << endl;
    }

    virtual ~Vehicle() = default;
};

class Bicycle : public Vehicle {
public:
    Bicycle(string r, double rate) : Vehicle(r, rate) {}

    double charge(int days) const override {
        return days * dailyRate;
    }

    void display() const override {
        cout << "Bicycle: " << regNo << endl;
    }
};
```

```
class MotorVehicle : public Vehicle {
protected:
    double level;                               // fuel or charge level
    Date serviceDue;                            // member class

public:
    MotorVehicle(string r, double rate, double lv, Date d)
        : Vehicle(r, rate), level(lv), serviceDue(d) {}
};

class TwoWheeler : public MotorVehicle {
public:
    TwoWheeler(string r, double rate, double lv, Date d)
        : MotorVehicle(r, rate, lv, d) {}

    double charge(int days) const override {
        return days * dailyRate;
    }

    void display() const override {
        cout << "Two Wheeler: " << regNo << endl;
    }
};

class Car : public MotorVehicle {
protected:
    int seats;

public:
    Car(string r, double rate, double lv, Date d, int s)
        : MotorVehicle(r, rate, lv, d), seats(s) {}

    double charge(int days) const override {
        return days * dailyRate;
    }

    void display() const override {
        cout << "Car: " << regNo
             << " Seats: " << seats << endl;
    }
};

class ElectricCar : public Car {
    double batteryKWh;
    double ratePerKWh;

public:
    ElectricCar(string r, double rate, double lv, Date d,
                int s, double kwh, double perKwh)
        : Car(r, rate, lv, d, s),
          batteryKWh(kwh), ratePerKWh(perKwh) {}

    double charge(int days) const override {
        return days * dailyRate + batteryKWh * ratePerKWh;
    }

    void display() const override {
```

```
        cout << "Electric Car: " << regNo
             << " Battery: " << batteryKWh << " kWh" << endl;
    }
};

// New vehicle type: added without changing any class above
class ElectricBus : public MotorVehicle {
    double batteryKWh;
    double ratePerKWh;

public:
    ElectricBus(string r)
        : MotorVehicle(r, 2500, 100, Date(1, 6, 2027)),
          batteryKWh(200), ratePerKWh(5) {}

    double charge(int days) const override {
        return days * dailyRate + batteryKWh * ratePerKWh;
    }

    void display() const override {
        cout << "Electric Bus: " << regNo << endl;
    }
};

int main() {
    vector<unique_ptr<Vehicle>> fleet;

    fleet.push_back(make_unique<Bicycle>("BIKE101", 200));
    fleet.push_back(make_unique<TwoWheeler>(
        "BIKE202", 300, 80, Date(15, 1, 2027)));
    fleet.push_back(make_unique<Car>(
        "CAR303", 1500, 60, Date(10, 2, 2027), 5));
    fleet.push_back(make_unique<ElectricCar>(
        "EV404", 2000, 90, Date(5, 3, 2027), 5, 75, 2));

    cout << "VEHICLE RENTAL FLEET\n\n";

    for (const auto& v : fleet) {
        v->display();
        cout << "Rental Charge for 2 days: "
             << v->charge(2) << endl;

        cout << "----------------------\n";
    }

    cout << "\nRENTAL OPERATIONS\n";

    fleet[0]->rent();
    fleet[0]->returnVehicle(2);

    cout << "\nADDING NEW VEHICLE TYPE\n";

    fleet.push_back(make_unique<ElectricBus>("BUS505"));

    fleet.back()->display();

    cout << "Rental Charge for 2 days: "
```

```
          << fleet.back()->charge(2) << endl;

    return 0;
}
```

Expected output:

```
VEHICLE RENTAL FLEET

Bicycle: BIKE101
Rental Charge for 2 days: 400
----------------------
Two Wheeler: BIKE202
Rental Charge for 2 days: 600
----------------------
Car: CAR303 Seats: 5
Rental Charge for 2 days: 3000
----------------------
Electric Car: EV404 Battery: 75 kWh
Rental Charge for 2 days: 4150
----------------------

RENTAL OPERATIONS
BIKE101 rented successfully
BIKE101 returned
Late Charge: 400

ADDING NEW VEHICLE TYPE
Electric Bus: BUS505
Rental Charge for 2 days: 6000
```

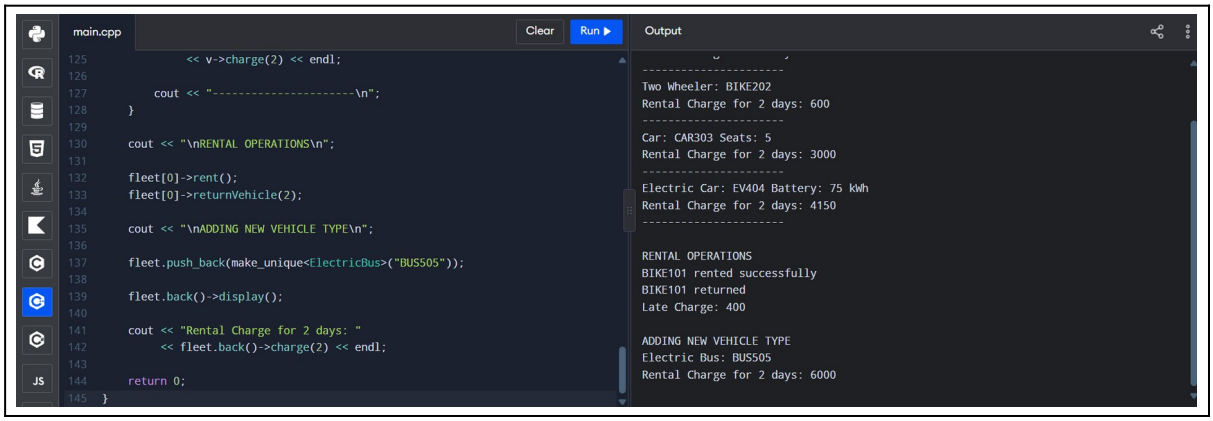


Figure 6.2: Output of the vehicle rental fleet program

Figure 6.3

Top part of the output (header, bicycle and first vehicles)

[Insert screenshot here: Task6_Fleet_Output_Top.png]

Figure 6.3: Top part of the program output

Figure 6.2 shows each vehicle type with its rental charge for 2 days: Two Wheeler BIKE202 (600), Car CAR303 with 5 seats (3000) and Electric Car EV404 with a 75 kWh battery (4150). In the rental operations, BIKE101 is rented successfully and returned with a late charge of 400. Finally a new vehicle type, Electric Bus BUS505, is added to the fleet with push_back and gives a charge of 6000 for 2 days. ElectricBus is a new class derived from MotorVehicle, and no existing class and no line of the existing loop was changed, which is possible because main() only uses Vehicle pointers and virtual calls.

6.8 Alternative Concept and Complexity Analysis

If the fleet used a single Vehicle class with a type code and a switch in charge(), instead of an abstract base class with virtual functions, the class would look like the code below.

Code / pseudocode of the alternative solution:

```
enum Kind { BICYCLE, TWO_WHEELER, CAR, ELECTRIC_CAR };

class Vehicle {
    Kind kind;
    string regNo; double dailyRate;
    int seats; double batteryKWh, ratePerKWh;   // unused for most kinds
public:
    double charge(int days) const {
        switch (kind) {
            case ELECTRIC_CAR:
                return days * dailyRate + batteryKWh * ratePerKWh;
            default:
                return days * dailyRate;
        }
    }
};
```

Aspect	Original solution	Alternative solution
Time per charge() call	O(1): one virtual call through the vtable.	O(1): one switch (compare / jump).
Space per object	Only the fields the type needs, plus one vtable pointer (8 bytes).	Every object holds the fields of all types (seats, batteryKWh, ratePerKWh) and a kind code, mostly unused.
Classes changed when a new type is added	None: one new class is written (ElectricBus).	The Vehicle class itself: the enum, the constructor, the switch in charge() and display(); all the other types must be retested.

6.9 Conclusion

A hierarchy with an abstract Vehicle class, a MotorVehicle level and a Date member class was designed from the requirements. The program rents and returns vehicles with a late charge and lists them in one loop, and the new Electric Bus type was added without changing any existing class.

Overall Learning Outcomes

Through this assignment I learned how to design a class hierarchy before writing code and how to separate the common behaviour (base class) from the behaviour that changes (virtual functions). I understood the difference between is-a and has-a relationships, the types of inheritance, and how virtual base classes remove the diamond problem. I traced the order of construction and destruction of base classes and member objects, and I saw how virtual functions, virtual destructors and base-class pointers decide which code runs and whether memory is released. I also learned to use the this pointer for chaining, to compare ways of reaching an object, and to compare each design with an alternative using Big-O analysis.

Final Conclusion

All six tasks were designed, implemented and executed with g++ -std=c++17 -Wall -Wextra. The repaired and redesigned programs avoid duplicated code, let new types be added without changing existing classes, and release their memory correctly. Writing and tracing the programs showed that a good hierarchy and correct use of virtual functions make a program reusable and easier to extend.

GitHub Repository Link

Repository link: https://github.com/192524062simats-ops/DSA0103c-/tree/main/c%2B%2B%20assignment

References

[1] Bjarne Stroustrup, The C++ Programming Language, 4th Edition, Addison-Wesley, 2013.

[2] cppreference.com, C++ reference documentation, https://en.cppreference.com

[3] Add your lecture notes, textbook or website references here.

[4]